RETRIEVAL-AUGMENTED GENERATION

Multi-Document RAG Knowledge Base

Build a local, privacy-first AI chatbot that answers questions from multiple documents — no cloud API required.

Ollama • llama3.2 | LangChain | ChromaDB | Streamlit | Lightning.ai

Mayank Chugh | Azure OpenAI & GenAI Architecture | 2026

What is RAG?

Retrieval-Augmented Generation — grounded answers from YOUR documents

❌ Traditional LLM

- Answers from training data
- May hallucinate facts
- No access to your documents
- Cannot cite sources

✅ RAG System

- Answers strictly from your docs
- Grounded — cannot hallucinate
- Retrieves relevant passages
- Always cites source & page

Key Insight: If the answer isn't in your documents, RAG says so — it never guesses.

Technology Stack

All components run locally — fully private, zero API cost

LLM

Ollama • llama3.2

Local inference — no OpenAI key needed

EMBEDDINGS

nomic-embed-text

Converts text chunks to semantic vectors

VECTOR DB

ChromaDB

Persistent vector store — one collection per doc

ORCHESTRATION

LangChain (LCEL)

Chains retrieval + prompt + LLM together

UI

Streamlit

Instant web chat interface, no frontend code

PLATFORM

Lightning.ai

Cloud GPU environment — free CPU tier available

Architecture — How It All Connects

PHASE 1 — INGESTION (Run Once)

1. **Upload PDF / DOCX**
   File saved to ./documents/
2. **Parse & Chunk**
   Split into 800-char chunks, 200 overlap
3. **Embed**
   nomic-embed-text → semantic vectors
4. **Store**
   ChromaDB — one collection per document

PHASE 2 — QUERY (Every Question)

1. **User Asks Question**
   Natural language query in chat
2. **Embed Query**
   Same model converts question to vector
3. **MMR Retrieval**
   fetch_k=20 → select best k=5 chunks
4. **Build Prompt**
   Chunks + history + question injected
5. **LLM Generates Answer**
   llama3.2 responds from context only

Step-by-Step Setup on Lightning.ai

1

Create a Studio

```
Go to lightning.ai → My Studios → New Studio → Blank (CPU — free)
```

2

Install Python Dependencies

```
pip install langchain langchain-community chromadb pypdf streamlit langchain-ollama
```

3

Install Ollama

```
curl -fsSL https://ollama.com/install.sh | sh
```

4

Pull Models & Start Ollama

```
ollama serve &   •   ollama pull llama3.2   •   ollama pull nomic-embed-text
```

5

Verify Ollama is Running

```
curl http://localhost:11434/api/tags    → you should see both models listed in JSON
```

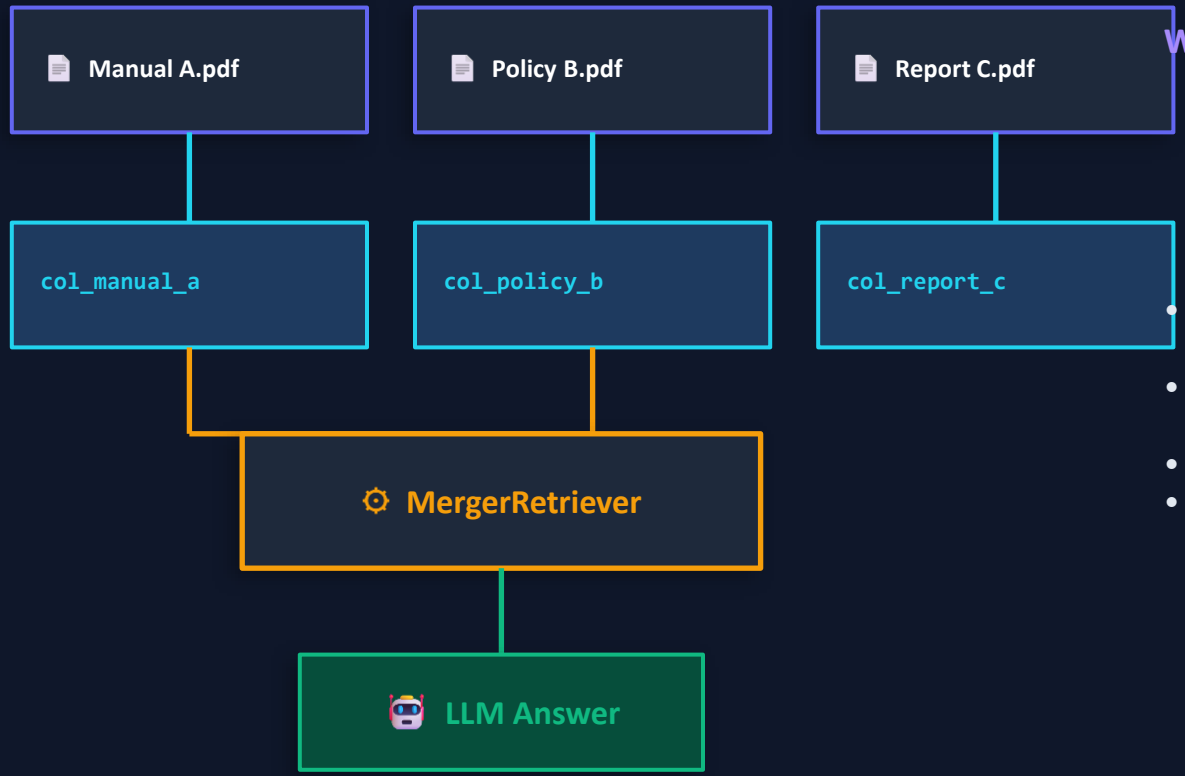
6

Upload App & Run

```
streamlit run app_multifile.py --server.port 8080
```

Multi-Collection Architecture

Each document gets its own isolated ChromaDB collection — mix and match at query time



Why This Design?

- Add new docs without affecting existing ones
- Delete one doc — only its collection removed
- Select any combo of docs per query
- MMR ensures diverse, non-redundant chunks

Key Features

📁 Multi-File Support

Upload any combo of PDFs, DOCX, TXT, CSV, images. Each gets its own collection.

🔍 MMR Retrieval

fetch_k=20, select k=5. Diverse results — no redundant chunks poisoning the answer.

📜 Source Citations

Every answer cites the source document and page number. Auditable and trustworthy.

⚡ Incremental Indexing

MD5 hash check — re-index only when file content actually changes.

🔒 Fully Local / Private

Zero data leaves your machine. No API keys. No cloud LLM billing.

📊 Show Retrieved Chunks

Toggle to inspect the exact chunks used to generate the answer — full transparency.

Configuration Reference

All constants live at the top of app_multifile.py — change once, works everywhere

CONSTANT	DEFAULT VALUE	DESCRIPTION
OLLAMA_BASE_URL	http://localhost:11434	Ollama API endpoint — change for remote server
LLM_MODEL	llama3.2	Swap to mistral, qwen2.5, etc for better quality
EMBED_MODEL	nomic-embed-text	Embedding model — must match indexed vectors
CHUNK_SIZE	800	Chars per chunk — smaller = precise, larger = context
CHUNK_OVERLAP	200	Overlap between chunks — prevents missed boundaries
TOP_K	5	Chunks fed to LLM — increase for complex questions
DOCS_DIR	./documents	Where uploaded files are stored locally
CHROMA_DIR	./chroma_db	ChromaDB persistence folder

MMR Retrieval — Why It Matters

❌ Plain Similarity Search

- Chunk 1: FWB structure overview
- Chunk 2: FWB structure details
- Chunk 3: FWB structure examples
- Chunk 4: FWB structure again
- Chunk 5: More FWB structure

Result: 5 near-identical chunks → redundant answers

✅ MMR (Max Marginal Relevance)

- Chunk 1: FWB structure overview
- Chunk 2: FWB field definitions
- Chunk 3: FWB message example
- Chunk 4: AWB relationship
- Chunk 5: Booking context

Result: 5 diverse chunks → richer, multi-angle answers

`Config: search_type = mmr | fetch_k = 20 (candidates) | k = 5 (final chunks sent to LLM)`

Model Selection Guide

Pick based on your hardware and quality requirements

Model	RAM	CPU Speed	T4 GPU Speed	Quality	Best For
`phi3:mini (3.8B)`	~3 GB	8–12 tok/s	60+ tok/s	Good	Quick testing
`llama3.2 (3B)`	~4 GB	6–10 tok/s	60+ tok/s	Good	Default choice ⭐
`mistral (7B)`	~8 GB	3–5 tok/s	40–50 tok/s	Better	Technical docs
`qwen2.5 (14B)`	~18 GB	1–2 tok/s	20–30 tok/s	Best local	Production use
`llama3 (70B)`	~45 GB	<1 tok/s	Needs A100	Excellent	Not practical locally

💡 Pro Tip: Do all setup on CPU (free). Switch to T4 GPU only when running inference — switch back to save credits.

Limitations & Production Upgrade Path

Current Limitations

- CPU inference: 4–8 tokens/sec on 7B models
- Complex PDFs with tables/images may parse poorly
- No authentication or multi-user support
- Context window capped at model's limit
- No hybrid (keyword + vector) search yet

Production Upgrade Path

P1 **Better Embeddings**

text-embedding-3-small → +20% accuracy

P2 **Add Reranker**

BGE-reranker-v2 → free, massive quality boost

P3 **Better PDF Parser**

Unstructured.io → handles tables & scanned PDFs

P4 **Upgrade LLM**

GPT-4o / Claude 3.5 → highest quality answers

What We Built — Live Demo

What the demo shows:

- ✓ Upload any PDF/DOCX → instantly indexed
- ✓ Ask any question in natural language
- ✓ Get cited answers with source + page
- ✓ Toggle to see exact retrieved chunks
- ✓ Select multiple docs → combined retrieval
- ✓ Response time shown (perf transparency)

Built with:

🦙 **Ollama + llama3.2**

100% local LLM — no API key

🔗 **LangChain LCEL**

Multi-retriever orchestration

🗄 **ChromaDB**

Persistent per-doc collections

⚡ **Lightning.ai**

Cloud GPU — free CPU tier

🖥 **Streamlit**

Zero-code web chat UI

Thanks for watching!

Multi-Document RAG Knowledge Base

Like & Subscribe

for more Azure OpenAI & GenAI content

Drop a comment

share what you'd like to RAG with!

Code on GitHub

link in description

Medium Article

full write-up with code walkthrough

Build once. Query anything. Stay private.